

Part 1. 코드 분석

① `prompts = [f'a photo of a {c}' for c in CIFAR10_CLASSES]`

역할: CIFAR-10의 각 클래스 이름을 CLIP이 이해하기 좋은 자연어 문장으로 바꾼다. 예를 들어 cat을 a photo of a cat처럼 만든다.

입출력 shape: 클래스 [10] → 문장 [10]

② `prompt_tokens = clip.tokenize(prompts).to(device)`

역할: 위의 프롬프트 문장들을 CLIP 텍스트 인코더에 넣을 수 있도록 토큰 텐서로 변환한다. CLIP은 기본적으로 문장 하나를 길이 77의 token sequence로 맞춘다. 또한 GPU를 사용할 수 있으면 GPU에서 연산하도록 한다.

입출력 shape: 문장 [10] → token [10, 77]

③ `class_features = model.encode_text(prompt_tokens)`

역할: 토큰화된 클래스 문장들을 CLIP text encoder에 넣어서 각 클래스의 feature vector로 만든다. 이것들이 나중에 이미지와 비교할 기준 벡터가 된다.

입출력 shape: token [10, 77] → feature vector [10, 512]

④ `class_features = class_features / class_features.norm(dim=-1, keepdim=True)`

역할: 각 클래스 embedding을 L2 정규화해서 벡터 길이를 1로 만든다. 이를 통해 dot product가 cosine similarity처럼 동작하게 한다.

입출력 shape: token vector [10, 512] → 정규화된 token vector [10, 512]

⑤ `img_feats = model.encode_image(batch_imgs)`

역할: batch size B 만큼의 CIFAR-10 이미지를 CLIP image encoder에 넣어서 이미지 feature vector로 만든다. CIFAR 이미지는 preprocess를 거쳐 CLIP 입력 크기인 224×224로 들어가고, 512차원으로 변환된다.

입출력 shape: 이미지 B개 [B, 3, 224, 224] → 이미지 feature vector [B, 512]

⑥ `img_feats = img_feats / img_feats.norm(dim=-1, keepdim=True)`

역할: 이미지 feature도 L2 정규화해서 각 이미지 벡터의 길이를 1로 만든다. 텍스트 feature와 같은 방식으로 정규화해야 안정적으로 similarity를 비교할 수 있다.

입출력 shape: 이미지 feature vector [B, 512] → 정규화된 이미지 feature vector [B, 512]

⑦ `sim = img_feats @ class_features.T`

역할: 각 이미지 feature와 각 클래스 text feature의 유사도를 계산한다. 결과적으로 배치 안의 모든 이미지에 대해 10개 클래스 점수가 나온다.

입출력 shape: 정규화된 이미지 feature vector [B, 512] @ text feature [512, 10] → [B, 10]

⑧ `preds = sim.argmax(dim=1).cpu()`

역할: 각 이미지마다 similarity가 가장 높은 클래스 index를 예측값으로 선택한다. dim=1은 10개 클래스 방향에서 최댓값을 고른다는 뜻이다.

입출력 shape: 10개 클래스 점수 [B, 10] → 최종 label B개 [B]

Part 2. 개념 질문

1. ④와 ⑥에서 L2 normalization을 수행한다. 이것을 생각하면 분류 결과에 어떤 영향을 줄 수 있는지 서술하시오.

④와 ⑥의 L2 normalization은 이미지 feature와 텍스트 feature의 벡터 길이를 1로 맞춰서, 이후의 dot product가 사실상 **cosine similarity**처럼 계산되도록 만든다. 즉, 벡터의 크기보다는 이미지와 클래스 문장의 방향적 유사도를 비교하게 된다.

이를 생각하면 feature의 방향뿐 아니라 **벡터 크기(norm)**도 점수에 영향을 주기 때문에, 실제 의미적으로 더 가까운 클래스가 아니라 feature vector 크기가 큰 클래스가 더 높은 점수를 받을 수 있다. 그래서 분류 결과가 불안정해지거나 특정 클래스 쪽으로 치우칠 수 있고, 전체 accuracy가 떨어질 수 있다.

다만 한 이미지 안에서 argmax를 할 때 이미지 feature의 norm은 모든 클래스 점수에 공통으로 곱해지는 값이라 영향이 비교적 작고, 특히 class feature의 norm 차이가 클래스 선택에 더 직접적인 영향을 줄 수 있다.

2. ⑦의 결과 텐서 sim의 shape이 (B, 10)이다. 이 행렬에서 하나의 행(row)이 의미하는 것과 하나의 열(column)이 의미하는 것을 각각 설명하시오.

sim은 배치 안의 이미지 feature와 CIFAR-10 클래스 text feature 사이의 유사도를 계산한 행렬이다.

하나의 **row**은 배치 안의 이미지 한 장에 대한 결과를 의미한다. 즉, 어떤 이미지 1장이 CIFAR-10의 10개 클래스 문장들과 각각 얼마나 유사한지를 나타내는 점수 10개가 들어 있다.

하나의 **column**은 특정 클래스 하나에 대한 결과를 의미한다. 예를 들어 cat 클래스에 해당하는 열이라면, 배치 안의 B개 이미지가 각각 a photo of a cat이라는 text feature vector와 얼마나 유사한지를 나타낸다.

이러한 row와 column이 모여 텐서 sim의 shape이 (B,10)이 된다.

3. 이 코드에는 CIFAR-10 이미지로 모델을 학습(training)하는 부분이 없다. 그럼에도 분류가 가능한 이유를 CLIP의 학습 방식과 연관지어 설명하시오.

이 코드에서는 CIFAR-10 이미지와 label을 이용해서 모델을 새로 학습하지 않는다. 대신 이미 학습된 CLIP 모델을 사용해서 이미지와 텍스트를 같은 embedding space에 배치한다.

CLIP은 사전에 약 4억 개라는 매우 많은 image-text pair를 이용해 학습된다. 학습 과정에서 서로 맞는 이미지와 문장은 embedding 공간에서 가깝게 만들고, 맞지 않는 이미지와 문장은 멀어지게 학습한다. 그래서 CLIP은 특정 데이터셋의 label에 직접 학습되지 않았더라도, 이미지와 자연어 문장 사이의 의미적 유사도를 계산할 수 있다.

이 코드에서는 CIFAR-10 클래스 이름을 a photo of a cat, a photo of a truck 같은 prompt로 바꾼 뒤, 이미지 embedding과 각 클래스 prompt embedding을 비교한다. 가장 유사도가 높은 prompt의 클래스를 예측값으로 선택하기 때문에, 별도의 CIFAR-10 training 없이도 zero-shot classification이 가능하다.

4. 이 코드의 zero-shot classification 구조와 수업 전반부에서 실습한 image-text retrieval 모델(Assignment #2)의 구조적 공통점과 차이점을 설명하시오.

두 구조의 공통점은 둘 다 CLIP의 image encoder와 text encoder를 사용한다는 점이다. 이미지는 image encoder를 통해 image embedding으로 바꾸고, 문장은 text encoder를 통해 text embedding으로 바꾼 뒤, 두 embedding 사이의 similarity를 계산한다. 또한 두 실습 모두 feature를 L2 normalization한 뒤 dot product를 사용해서 cosine similarity처럼 비교한다.

차이점은 similarity를 사용한 뒤의 목적이 다르다는 점이다. image-text retrieval에서는 여러 이미지와 여러 텍스트 사이의 similarity matrix를 만든 뒤, 어떤 이미지와 어떤 텍스트가 가장 잘 맞는지 찾는다. 즉, 주된 목적은 이미지 기준으로 관련 텍스트를 찾거나, 텍스트 기준으로 관련 이미지를 찾는 것이다.

반면 zero-shot classification에서는 텍스트 후보가 일반 문장이 아니라 CIFAR-10의 클래스 prompt 10개로 고정되어 있다. 각 이미지에 대해 10개 클래스 prompt와의 유사도를 계산한 뒤, 가장 높은 점수를 가진 클래스를 선택한다. 그래서 retrieval은 Image-Text 검색 문제에 가깝고, zero-shot classification은 텍스트 prompt를 label처럼 사용한 분류 문제에 가깝다.

Part 3. Custom Query Retrieval

5개 query와 이에 대한 retrieval 결과 캡처

1. a dog running on grass



2. a group of people standing outside



3. a child playing with a ball



4. a man riding a horse



5. a bird flying in the sky



retrieval이 잘 된 사례 2개

1. a dog running on grass



잘 된 이유: a dog running on grass는 **dog**와 **grass**가 이미지에서 모두 뚜렷하게 보이고, 대부분의 결과가 실제로 잔디 위에 있는 개를 보여주기 때문에 retrieval이 잘 된 사례로 볼 수 있다. 특히 개가 화면의 중심에 크게 나타나 있어서 query의 핵심 객체와 이미지가 잘 대응된다.

2. a group of people standing outside



잘 된 이유: a group of people standing outside는 여러 사람이 함께 서 있는 장면과 야외 배경이 검색 결과에 잘 나타나서 query와 잘 맞는다. 사람의 수나 자세는 이미지마다 조금 다르지만, 전체적으로 “야외에 있는 사람들”이라는 의미는 잘 반영된 것으로 보인다.

retrieval 이 잘 안 된 사례 1 개

1. a bird flying in the sky



잘 안 된 이유: a bird flying in the sky는 새가 나오는 사진이 2개 존재하지만, 첫 번째 결과는 새가 아니라 스키를 타고 있는 사람이 담긴 이미지였고, 4번째와 5번째 사진도 서핑하는 사람과 그네 타는 아이이 담겨 있었다. 즉 query와 맞지 않는 결과가 섞였다. 새가 작게 보이거나 하늘 배경이 더 강하게 반영되면서, CLIP이 "bird"보다 "sky/flying"과 비슷한 장면을 더 넓게 가져온 것으로 보인다.